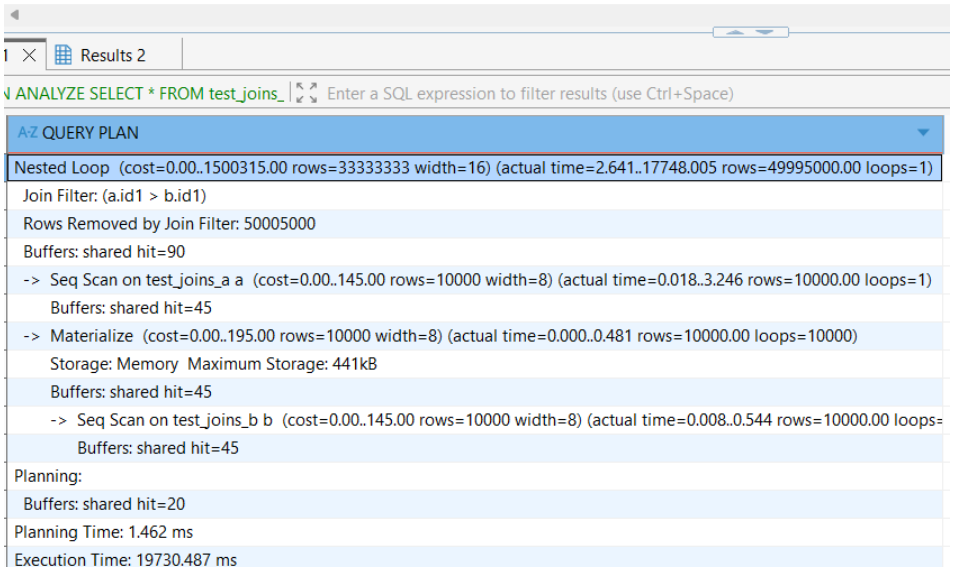
--TASK 1 NESTED LOOP JOIN

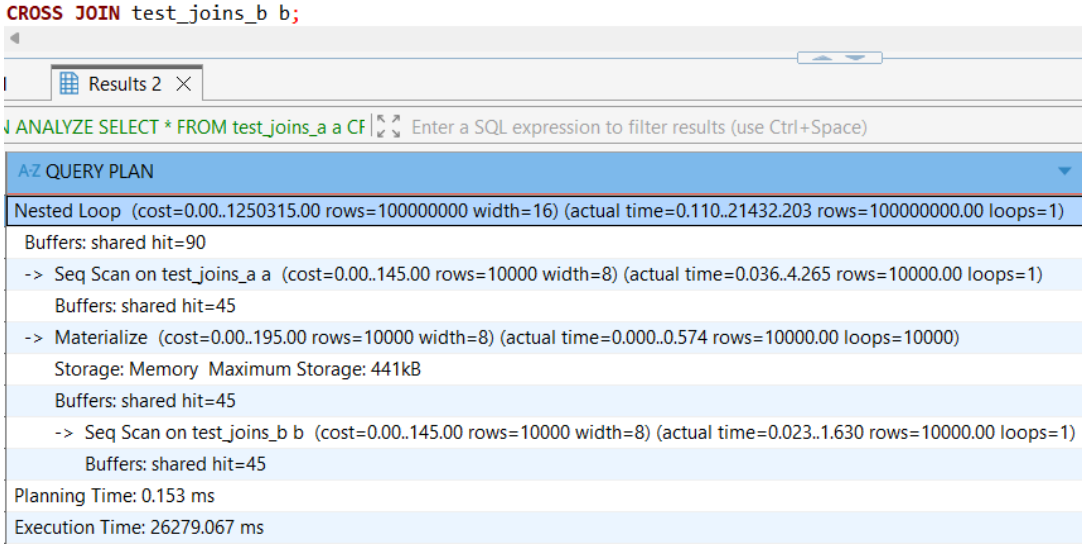
Nested Loop is the only option for inequality condition for such a query. A hash and merge require search on equal conditions

```
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 > b.id1;
```



The second is an explicit cross join. Every row of A pairs with every row of B, so there's no join condition at all to hash or sort on. Nested Loop is structurally the only way to produce full combination.

```
EXPLAIN ANALYZE
SELECT *
FROM test_joins_a a
CROSS JOIN test_joins_b b;
```



--TASK 2 HASH JOIN

The hash join is applied after the > is changed to = . One of the tables is hashed to the memory(Buckets: 16384 Batches: 1 Memory Usage: 519kB). This is faster than nested loops because the one table scanned through the hash table once.

```
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 = b.id1;
```

s 1

IN ANALYZE SELECT * FROM test_joins_a a, tes | Enter a SQL expression to filter results (use Ctrl+Space)

QUERY PLAN
Hash Join (cost=270.00..552.50 rows=10000 width=16) (actual time=1.780..4.814 rows=10000.00 loops=1)
Hash Cond: (a.id1 = b.id1)
Buffers: shared hit=90
-> Seq Scan on test_joins_a a (cost=0.00..145.00 rows=10000 width=8) (actual time=0.017..0.531 rows=10000.00 loops=1)
Buffers: shared hit=45
-> Hash (cost=145.00..145.00 rows=10000 width=8) (actual time=1.683..1.684 rows=10000.00 loops=1)
Buckets: 16384 Batches: 1 Memory Usage: 519kB
Buffers: shared hit=45
-> Seq Scan on test_joins_b b (cost=0.00..145.00 rows=10000 width=8) (actual time=0.008..0.518 rows=10000.00 loops=
Buffers: shared hit=45
Planning:
Buffers: shared hit=28
Planning Time: 0.253 ms
Execution Time: 5.238 ms

WHERE EXISTS map to semi joins, which return each row of the left table once regardless of how many matches exist on the right side. Hash Join makes sense here since each id1 in a has exactly one match in

```
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a
WHERE EXISTS (
    SELECT 1 FROM test_joins_b b WHERE b.id1 = a.id1
);
```

1 | Results 2 | Results 3 | Results 4 | Statistics 4 | Results 5

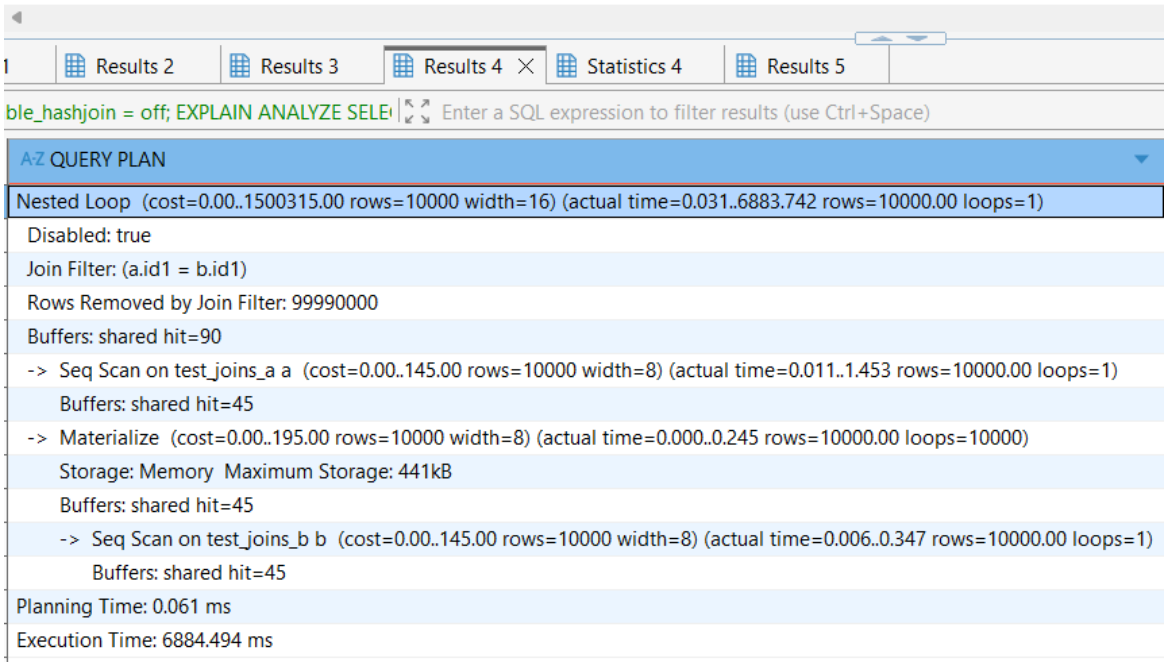
N ANALYZE SELECT * FROM test_joins_a a W | Enter a SQL expression to filter results (use Ctrl+Space)

QUERY PLAN
Hash Semi Join (cost=270.00..552.50 rows=10000 width=8) (actual time=1.445..3.123 rows=10000.00 loops=1)
Hash Cond: (a.id1 = b.id1)
Buffers: shared hit=90
-> Seq Scan on test_joins_a a (cost=0.00..145.00 rows=10000 width=8) (actual time=0.013..0.358 rows=10000.00 loops=1)
Buffers: shared hit=45
-> Hash (cost=145.00..145.00 rows=10000 width=4) (actual time=1.380..1.380 rows=10000.00 loops=1)
Buckets: 16384 Batches: 1 Memory Usage: 480kB
Buffers: shared hit=45
-> Seq Scan on test_joins_b b (cost=0.00..145.00 rows=10000 width=4) (actual time=0.006..0.535 rows=10000.00 loops=1)
Buffers: shared hit=45
Planning Time: 0.372 ms
Execution Time: 3.460 ms

b.

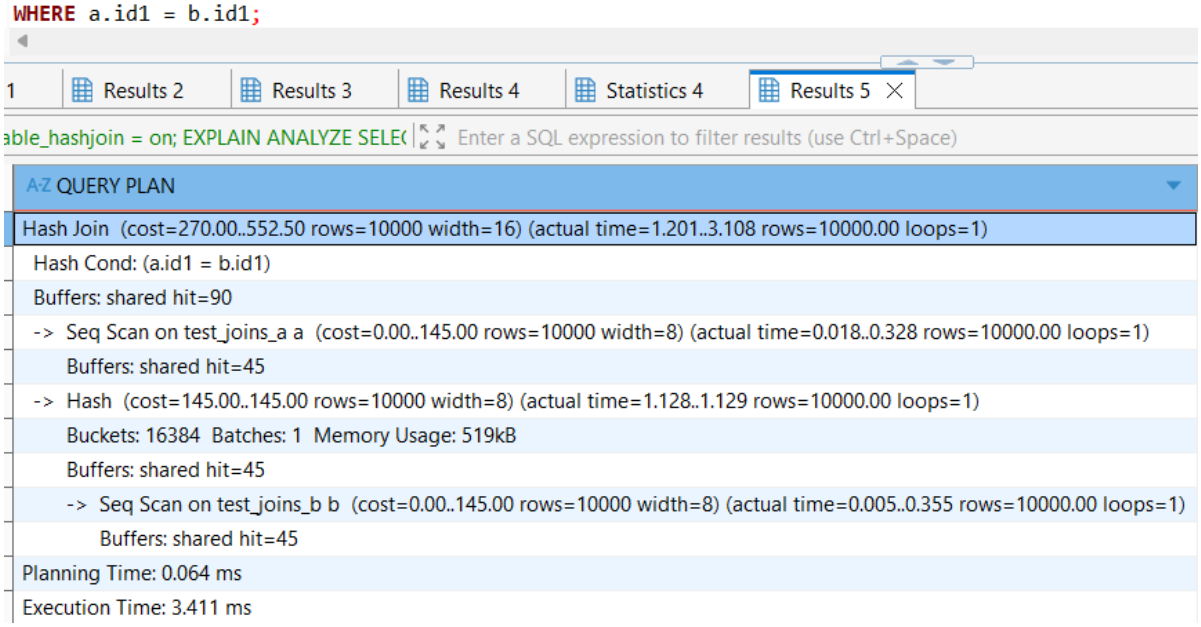
Turning off enable_hashjoin shifts the query to Nested Loop that process the query, but taking 6885ms instead of 3 ms, significantly slower, because now it has to check all combinations and filter down instead of doing a direct hash lookup.

```
SET enable_hashjoin = off;
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 = b.id1;
```



Turning on enable_hashjoin back shows quick results(3ms)

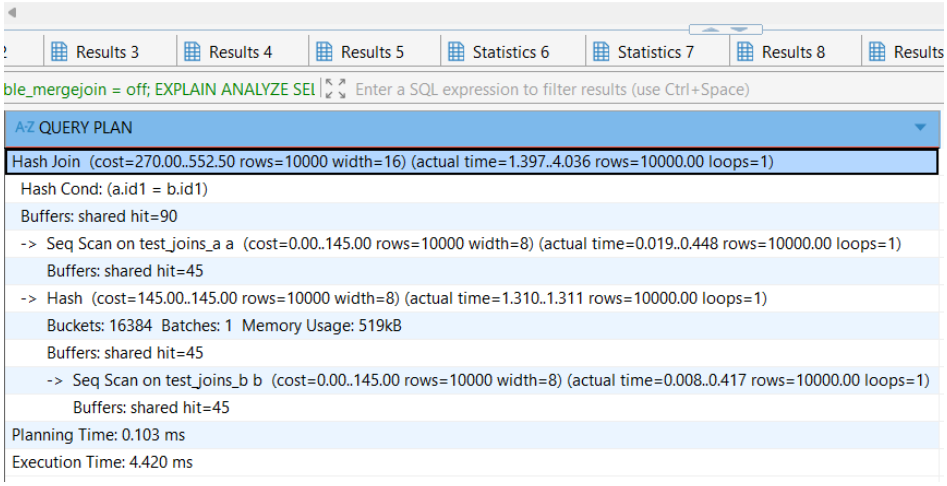
```
SET enable_hashjoin = on;
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 = b.id1;
```



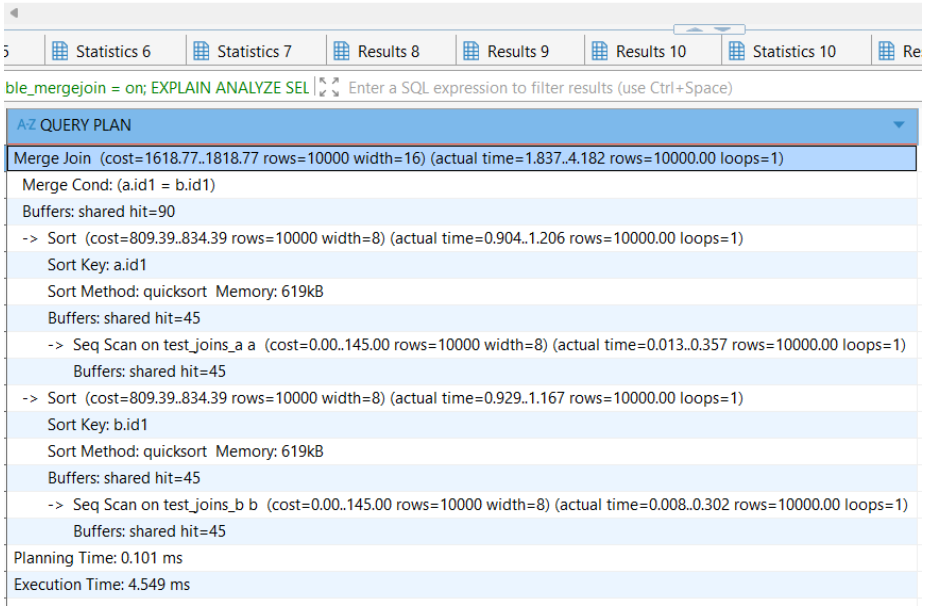
--TASK 3 MERGE JOIN

The Hash Join is still preferable in equal conditions, merge is preferable when hash is off only, the postgres needs extra time to sort tables.

```
SET enable_mergejoin = off;
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 = b.id1;
```




```
SET enable_mergejoin = on;
EXPLAIN ANALYZE
SELECT * FROM test_joins_a a, test_joins_b b
WHERE a.id1 = b.id1;
```

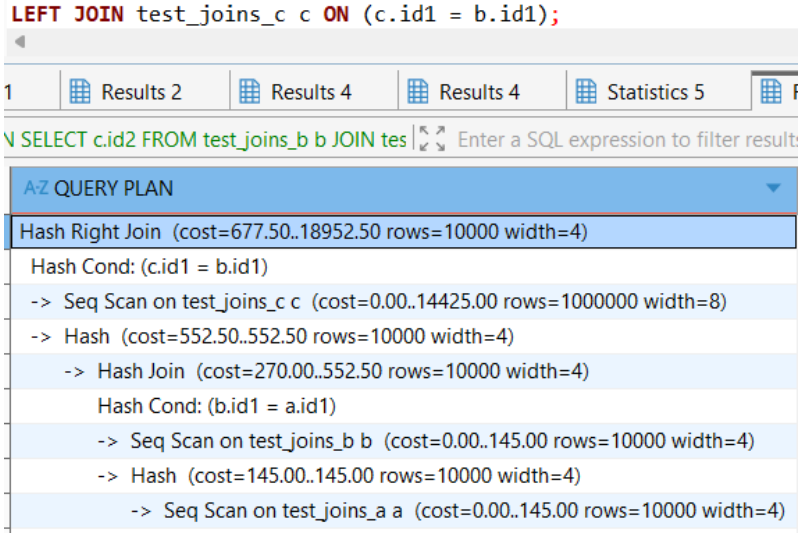


-- TASK 4 CHANGING JOIN ORDER

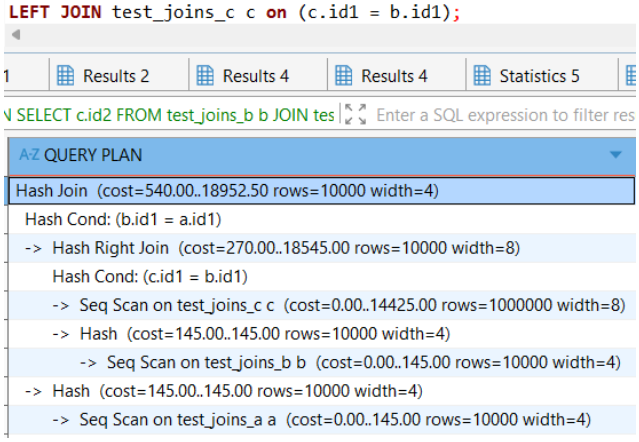
Forcing join_collapse_limit = 1 reflects the literal order: b JOIN a first, then joined to c.(hash right join) With the default join_collapse_limit = 8, Postgres reordered the joins differing from the literal query order (b JOIN a, then LEFT JOIN c)(hash join)

```
SET join_collapse_limit = 1;

EXPLAIN
SELECT c.id2
FROM test_joins_b b
JOIN test_joins_a a ON (b.id1 = a.id1)
LEFT JOIN test_joins_c c ON (c.id1 = b.id1);
```




```
EXPLAIN
SELECT c.id2
FROM test_joins_b b
JOIN test_joins_a a on (b.id1 = a.id1)
LEFT JOIN test_joins_c c on (c.id1 = b.id1);
```



--TASK 5 LATERAL JOIN

```sql
SELECT
    s.store_id,
    s.store_name,
    o.order_id,
    o.order_num,
    o.order_cost
FROM stores s
CROSS JOIN LATERAL (
    SELECT
        order_id,
        order_num,
        order_cost
    FROM orders
    WHERE order_cost <= s.max_order_cost
    ORDER BY order_cost DESC
    LIMIT 10
) o
ORDER BY
    s.store_id,
    o.order_cost DESC;
```

ults 1 | Results 2 | Results 4 | Results 4 | Statistics 5 | stores(+) 6 | Statistics 7

ECT s.store_id, s.store_name, o.order_id, o.order_num, o.order_ Enter a SQL expression to filter results (use Ctrl+Space

123 store_id	A-Z store_name	123 order_id	A-Z order_num	123 order_cost
1	grossery shop	11	order number 11	776
1	grossery shop	20	order number 20	763
1	grossery shop	379	order number 379	745
1	grossery shop	15	order number 15	744
1	grossery shop	950	order number 950	729
1	grossery shop	180	order number 180	729
1	grossery shop	139	order number 139	717
1	grossery shop	106	order number 106	701
1	grossery shop	18	order number 18	700
1	grossery shop	428	order number 428	676
2	bakery	3	order number 3	71
2	bakery	1	order number 1	50
2	bakery	2	order number 2	42
2	bakery	21	order number 21	41
2	bakery	4	order number 4	3
3	manufactured goods	38	order number 38	2,961
3	manufactured goods	31	order number 31	2,960
3	manufactured goods	60	order number 60	2,955
3	manufactured goods	152	order number 152	2,849
3	manufactured goods	67	order number 67	2,843
3	manufactured goods	346	order number 346	2,763
3	manufactured goods	70	order number 70	2,750
3	manufactured goods	372	order number 372	2,679
3	manufactured goods	393	order number 393	2,656
3	manufactured goods	888	order number 888	2,655

Due to lateral join type, the FROM subquery runs once per row of stores, each time substituting that store's own max_order_cost, and returns up to 10 matching orders per store.
This resembles for each loop approach and allows to scan table for required values.

--TASK 6 RECURSIVE CTE

```sql
WITH RECURSIVE emp_hierarchy AS (
    SELECT empno, mgr, ename, ename AS mgrname, 1 AS lvl
    FROM emp
    WHERE mgr IS NULL
    UNION ALL
    SELECT e.empno,e.mgr,e.ename,h.ename AS mgrname,h.lvl + 1 AS lvl
    FROM emp e
    JOIN emp_hierarchy h ON e.mgr = h.empno
)
SELECT empno, mgr, ename, mgrname, lvl
FROM emp_hierarchy
ORDER BY lvl, empno;
```

ults 1 | Results 2 | Results 4 | Results 4 | Statistics 5 | emp_hierarch

'H RECURSIVE emp_hierarchy AS (SELECT empr | Enter a SQL expression to filter results (use Ctrl+S

123 empno	123 mgr	A-Z ename	A-Z mgrname	123 lvl
9	[NULL]	KING	KING	1
4	9	JONES	KING	2
6	9	BLAKE	KING	2
7	9	CLARK	KING	2
2	6	ALLEN	BLAKE	3
3	6	WARD	BLAKE	3
5	6	MARTIN	BLAKE	3
8	4	SCOTT	JONES	3
10	6	TURNER	BLAKE	3
12	6	JAMES	BLAKE	3
13	4	FORD	JONES	3
1	13	SMITH	FORD	4
11	8	ADAMS	SCOTT	4
14	1	KELLY	SMITH	5
15	1	HANNAH	SMITH	5

Firstly, the president (mgr IS NULL) was selected with assigning lvl = 1.
Next, joins emp to the emp_hierarchy - find employees whose manager is someone we already placed in the hierarchy.
Query keeps re-running against the newly added rows from the previous pass, stopping automatically once a pass returns zero new rows.

--TASK 7 CHANGING DATA CTE

```sql
WITH updated_rows AS (
    UPDATE orders
    SET order_cost = order_cost / 2
    WHERE order_cost BETWEEN 100 AND 1000
    RETURNING order_id, order_cost, order_num
),
deleted_rows AS (
    DELETE FROM orders
    WHERE order_cost < 50
    RETURNING order_id, order_cost, order_num
)
INSERT INTO order_log (order_id, order_cost, order_num, action_type)
SELECT order_id, order_cost, order_num, 'U' FROM updated_rows
UNION ALL
SELECT order_id, order_cost, order_num, 'D' FROM deleted_rows;
```

ılts 1 | Results 2 | Results 4 | Results 4 | Statistics 5 | order_log 6 | orders 9

CT * FROM order_log ORDER BY log_date | Enter a SQL expression to filter results (use Ctrl+Space)

log_id	order_id	order_cost	order_num	action_type	log_date
39	451	209	order number 451	U	2026-07-14 22:25:15.973 +0300
40	503	301	order number 503	U	2026-07-14 22:25:15.973 +0300
41	561	70	order number 561	U	2026-07-14 22:25:15.973 +0300
42	653	270	order number 653	U	2026-07-14 22:25:15.973 +0300
43	712	405	order number 712	U	2026-07-14 22:25:15.973 +0300
44	941	119	order number 941	U	2026-07-14 22:25:15.973 +0300
45	950	364	order number 950	U	2026-07-14 22:25:15.973 +0300
46	979	488	order number 979	U	2026-07-14 22:25:15.973 +0300
47	2	42	order number 2	D	2026-07-14 22:25:15.973 +0300
48	4	3	order number 4	D	2026-07-14 22:25:15.973 +0300
49	21	41	order number 21	D	2026-07-14 22:25:15.973 +0300

The WITH statement combines an UPDATE (change cost for rows 100–1000) and a DELETE (remove rows with cost < 50), both with RETURNING, adding into an INSERT that logs both actions to order_log with action type 'U'/'D'.